# Implementation of a Secure AI-Integrated Digital Banking System Using React, Node.js, PostgreSQL, and FastAPI


## Abstract

Digital banking systems have become an essential component of modern financial infrastructure; however, they continue to face challenges related to transaction security, fraud detection, scalability, and secure authentication. This paper presents the design and implementation of SecureBank, a secure full-stack digital banking platform integrated with an AI-based fraud detection microservice. The proposed system combines a React.js frontend, Node.js backend services, PostgreSQL database management, blockchain-supported QR transaction mechanisms, and a FastAPI-based machine learning service utilizing the LightGBM algorithm for fraud detection. The system implements secure authentication, encrypted credential storage, transaction orchestration, and intelligent risk scoring for transaction validation. A modular microservice architecture was adopted to improve scalability and maintainability. Experimental evaluation demonstrated that the proposed system successfully performs secure transaction handling, detects suspicious activities using AI-driven classification, and provides an efficient user experience.



**Keywords:** Digital Banking, Fraud Detection, FastAPI, React.js, Node.js, PostgreSQL, LightGBM, Blockchain, Microservices, Cybersecurity


---

# I. Introduction

The rapid growth of digital payment systems and online banking platforms has significantly transformed the global financial ecosystem. Modern banking applications provide users with seamless access to financial services such as money transfers, account management, digital payments, and transaction monitoring. However, the increasing adoption of digital financial services has also resulted in a substantial rise in cybersecurity threats including account hijacking, phishing attacks, unauthorized transactions, identity theft, and fraudulent payment activities.

Traditional banking systems often rely on centralized transaction processing architectures and rule-based fraud detection mechanisms that struggle to adapt to rapidly evolving cyber threats. Moreover, many existing systems suffer from limitations such as poor scalability, inadequate authentication mechanisms, delayed fraud detection, and inefficient transaction monitoring.

To address these challenges, this paper presents the implementation of SecureBank, a secure AI-integrated digital banking platform developed using React.js, Node.js, PostgreSQL, FastAPI, and LightGBM. The system combines secure transaction orchestration, QR-based payment mechanisms, blockchain-assisted transaction verification, and intelligent fraud detection to provide a modern digital banking solution.

---

# II. Problem Statement

Despite the widespread adoption of online banking systems, several security and operational limitations continue to affect modern financial applications. Many traditional systems rely on static rule-based fraud detection approaches that fail to identify evolving attack patterns and sophisticated financial fraud techniques.

Major problems identified in existing systems include:

1. Weak authentication mechanisms.
2. Delayed fraud detection.
3. Poor scalability in monolithic architectures.
4. Insecure transaction handling.
5. Limited real-time transaction analysis.
6. Lack of intelligent risk assessment.

The SecureBank platform was implemented to provide secure authentication, AI-driven fraud analysis, scalable microservice architecture, and reliable transaction processing.

---

# III. Objectives

The objectives of the proposed system are:

1. To implement a secure digital banking platform.
2. To integrate machine learning-based fraud detection.
3. To provide QR-based payment functionality.
4. To ensure secure transaction orchestration.
5. To improve transaction monitoring and security logging.
6. To design a scalable microservice-based architecture.

---

# IV. Literature Review

Recent research in digital banking security has focused on fraud detection systems, secure authentication mechanisms, blockchain-assisted payment verification, and scalable backend architectures. Machine learning algorithms such as Decision Trees, Random Forest, XGBoost, and LightGBM have demonstrated promising results in identifying fraudulent financial transactions.

Traditional fraud detection systems primarily rely on predefined rules and static thresholds, which often fail to adapt to emerging fraud patterns. Recent studies indicate that AI-driven fraud detection systems can significantly improve detection accuracy by analyzing transaction patterns and user behavior.

Modern banking platforms increasingly adopt microservice architectures to improve scalability and maintainability. Blockchain-assisted QR payment systems also provide enhanced transparency and secure transaction verification.

However, many existing implementations focus only on isolated fraud detection pipelines without integrating complete secure banking functionality. Therefore, SecureBank was developed as a unified platform integrating frontend banking operations, secure backend transaction processing, AI-based fraud detection, and database security.

---

# V. System Architecture

The SecureBank system follows a modular microservice architecture consisting of four major components:

1. React.js frontend client.
2. Node.js backend transaction service.
3. FastAPI fraud detection microservice.
4. PostgreSQL database system.

## A. User Management Module

The user management module handles account registration, authentication, credential validation, and user session management. Passwords are securely encrypted using hashing mechanisms before storage.

## B. QR Transaction Module

Users can generate and scan QR codes to perform digital transactions securely. QR-based payments improve usability and transaction efficiency.

## C. Transaction Orchestration Module

The Node.js backend validates transaction requests, verifies balances, coordinates transaction flows, and communicates with the fraud detection service.

## D. AI Fraud Detection Module

The FastAPI microservice uses the LightGBM machine learning model to generate fraud risk scores based on transaction features. Transactions identified as suspicious are automatically blocked.

---

# VI. Technology Stack

| Component | Technology |
|---|---|
| Frontend | React.js |
| Backend | Node.js, Express.js |
| Database | PostgreSQL |
| Machine Learning | FastAPI |
| Fraud Detection Model | LightGBM |
| Authentication | JWT |
| Password Security | bcrypt |
| API Communication | REST APIs |
| Version Control | GitHub |

---

# VII. Database Design

The SecureBank database was implemented using PostgreSQL relational tables.

## A. Users Table

The Users table stores:

- User ID
- Username
- Password hash
- UPI ID
- Account balance
- Contact information

Passwords are stored in encrypted hash format for security.

## B. Transaction Tables

Transaction tables maintain records for sent and received transactions. Each transaction stores:

- Sender and receiver identifiers
- Transaction amount
- Fraud score

- Fraud status
- Timestamp

### C. Security Logs

The Security_Logs table records login activities, suspicious events, and transaction operations for monitoring and auditing.

### D. Fraud Training Dataset

The FraudTrainingData table stores transactional features used for training the fraud detection model.

---

# VIII. Implementation Methodology

The implementation process was divided into several development phases.

### Phase 1: Requirement Analysis

System requirements were identified through literature surveys and analysis of existing banking security limitations.

### Phase 2: Dataset Preprocessing

The PaySim dataset was used for training the fraud detection model. Preprocessing included:

- Feature extraction
- Data cleaning
- Missing value handling
- Transaction ratio calculations

### Phase 3: Backend Development

REST APIs were developed using Node.js and Express.js for:

- User authentication
- Transaction processing
- Fraud verification
- Balance updates
- Database communication

## Phase 4: Machine Learning Integration

A FastAPI microservice was developed for AI-based fraud detection. Transaction feature vectors were transmitted asynchronously to the fraud detection service.

The LightGBM model analyzed:

- Transaction amount
- Old account balance
- New account balance
- Balance ratio changes

## Phase 5: Frontend Development

The frontend interface was developed using React.js and includes:

- Login and registration pages
- Transaction dashboard
- QR transaction scanner
- Balance monitoring
- Fraud alerts

## Phase 6: System Integration and Testing

All modules were integrated and tested through end-to-end transaction simulations.

---

# IX. Transaction Workflow

The transaction workflow operates as follows:

1. The user scans a QR code.
2. The frontend submits the transaction request.
3. The backend validates transaction details.
4. Transaction feature vectors are generated.
5. The FastAPI fraud detection service evaluates the transaction.
6. A fraud score is returned.
7. Transactions below the threshold are committed.
8. Suspicious transactions are blocked automatically.

This workflow enables real-time fraud prevention before transaction completion.

---

# X. PERT Development Analysis

Project planning was performed using a Program Evaluation and Review Technique (PERT) model.

The major development stages included:

- Requirement analysis
- Dataset preprocessing
- Model training
- Backend development
- Frontend development
- Smart contract integration
- System integration
- Final testing and documentation

The critical path was identified to optimize development scheduling and implementation efficiency.

---

# XI. Results and Evaluation

The SecureBank platform was successfully implemented and evaluated.

## A. Functional Testing

The following functionalities were tested successfully:

- User registration and login
- QR code transactions
- Fraud score generation
- Transaction blocking
- Balance updates
- Security logging

## B. Fraud Detection Performance

The LightGBM model successfully identified suspicious transactions based on transaction features and account balance variations.

## C. Security Evaluation

Security mechanisms implemented include:

- Password hashing
- JWT authentication

- Secure API validation
- Fraud risk analysis
- Transaction monitoring
- Database security logs

### D. Scalability

The microservice architecture improved scalability and modularity by separating fraud detection services from core banking operations.

---

# XII. Challenges Faced

Several technical challenges were encountered:

1. Synchronization between backend services and fraud analysis.
2. Transaction rollback handling.
3. Integration of FastAPI with Node.js.
4. Database consistency during concurrent transactions.
5. Secure authentication synchronization.
6. Feature engineering for fraud detection.

These issues were addressed through modular architecture and asynchronous communication mechanisms.

---

# XIII. Future Scope

Future improvements may include:

1. Blockchain-based distributed ledgers.
2. Biometric authentication.
3. Deep learning fraud detection models.
4. Kubernetes-based deployment.
5. Real-time analytics dashboards.
6. Mobile banking application support.
7. Advanced anomaly detection systems.

---

# XIV. Conclusion

This paper presented the implementation of SecureBank, a secure AI-integrated digital banking platform developed using React.js, Node.js, PostgreSQL, FastAPI, and LightGBM. The system integrates secure

authentication, QR-based payment processing, AI-driven fraud detection, and modular microservice architecture.

The implementation demonstrates how machine learning can be integrated into real-time banking systems to improve transaction security and fraud prevention. The proposed architecture improves scalability, maintainability, and transaction reliability while providing a practical and secure digital banking solution.

The project highlights the importance of combining cybersecurity principles, machine learning techniques, and modern web technologies to develop next-generation intelligent financial systems.

---

# References


1. Research papers on AI-based fraud detection systems.
2. Studies on secure digital banking architecture.
3. Documentation for React.js, Node.js, PostgreSQL, FastAPI, and LightGBM.
4. PaySim financial transaction dataset documentation.
5. Literature on blockchain-assisted payment systems.
6. Cybersecurity studies related to digital banking platforms.